

Logical-JEPA — Completion Guide

Step-by-step, manual instructions: finish and push from this laptop, then run every remaining GPU experiment on a teammate's laptop and bring the results back.

Item	Value
Repository	https://github.com/Ratnachand04/logical-JEPA.git
Working branch	vikram (all Phase 3b/3c work; main has only the first commit, via PR #1)
Git identity to use	vikram5002 <bunny.vikram18@gmail.com>
Latest pushed commit	3ad6f84 — Show the cardinality channel in the web demo
This laptop	D:\cv_prj, GTX 1650 4 GB, Python 3.12, PyTorch 2.14+cu126 in D:\pylibs
Shell used below	Windows PowerShell unless a block says otherwise

0. Where the project stands

All code work is finished and pushed. What remains is GPU time and a write-up of what it measures.

Phase	State	Result
1, 2, 3a, 5	Done (before this round)	See outputs/phase*/SUMMARY.md
3b hierarchical slot loss	Done, measured (3 seeds)	Negative: slot term only lowers AUROC (0.806 → 0.72–0.73 as λ grows)
3c cardinality head	Done, measured (3 seeds)	Negative: near chance alone (0.541); hurts localisation when added
Per-direction breakdown	Done	Removed objects 0.47–0.58, added/moved ~1.0 in every arm
Web demo cardinality card	Done	Shown when run with anomaly.cardinality=add only
fp32 target_std check	Pending (GPU)	Part B, step B5
Real MVTec LOCO studies	Pending (GPU + dataset)	Part B, steps B6–B10
Removal-scoring research	Pending (research)	Part B, step B11
Merge vikram → main	Partly done	PR #1 merged 81f6062 into main. 5cbd3bb (results + loading fix) and 3ad6f84 (demo card) still need a second PR: Part C, step C3

Part A — On this laptop (push everything to git)

Everything is already pushed as of commit **3ad6f84**. Use this part to verify that, and any time you change something here later.

A1. Open PowerShell in the project and switch to the work branch

This laptop is currently checked out on **main** at an old commit (2c0fd43), so the Phase 3b/3c files are not visible on disk. Nothing is lost: all of it is on the local branch **phase3bc-slots-cardinality** and on the remote **vikram** branch. Switch back first:

```
cd D:\cv_prj
git status                                # must be clean before switching
git checkout phase3bc-slots-cardinality
git log --oneline -3                     # expect 3ad6f84 at the top
dir models\cardinality.py                # the Phase 3c file is back
```

- **git status** should say *nothing to commit, working tree clean*.
- Branch **phase3bc-slots-cardinality** pushes to the remote **vikram** branch (A4).
- To refresh your local **main** with the owner's merge of PR #1: **git checkout main; git pull origin main**, then switch back as above.

A2. Make sure commits are authored as vikram5002

This repo is already configured locally. Check, and set it if it is ever lost:

```
git config user.name
git config user.email
# only if the two lines above are not vikram5002 / bunny.vikram18@gmail.com:
git config user.name "vikram5002"
git config user.email "bunny.vikram18@gmail.com"
```

A3. Commit new changes (only if A1 showed any)

```
git add <the files you changed>
git status
git commit -m "Short description of the change"
```

Never commit checkpoints/, data/, outputs/* (except **outputs/<phase>/SUMMARY.md**), or **rmwo/**. They are large, regenerable or private. **.gitignore** already blocks the first three and **rmwo/** is excluded locally, so **git status** should never list them. Add files by name, not with **git add .**

A4. Push to the vikram branch

```
git fetch origin
git push origin HEAD:vikram
git log origin/vikram --oneline -3
```

- The last command should show your newest commit at the top.
- If the push is **rejected (non-fast-forward)**, someone else pushed to vikram. Run **git pull --rebase origin vikram**, re-run the tests, then push again. Do **not** force-push over a teammate's work.

A5. Run the tests before every push

```
py -3.12 -m pytest tests/ -q
```

Expected: **285 passed**. A SciPy/NumPy version warning is harmless.

A6. Free the GPU on this laptop

```
tasklist | findstr python
taskkill /F /T /PID <PID shown above>    # only for a training/evaluation run you started
```

Part B — On the teammate's laptop (all GPU work)

Needs: Windows, an NVIDIA GPU with a recent driver, Git, Python 3.12, and roughly 15 GB free disk (PyTorch with CUDA is ~4.5 GB; real LOCO adds ~6 GB). The project deliberately uses the global Python 3.12, not a virtual environment.

B1. Get the code on the vikram branch

First time:

```
cd C:\work # any folder with enough space
git clone https://github.com/Ratnachand04/logical-JEPA.git
cd logical-JEPA
git checkout vikram
git log --oneline -1 # expect 3ad6f84 or newer
```

Already cloned before:

```
cd C:\work\logical-JEPA
git fetch origin
git checkout vikram
git reset --hard origin/vikram # discards local edits on this branch - commit them first if needed
```

B2. Install dependencies

```
py -3.12 -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
py -3.12 -m pip install -r requirements.txt pytest
py -3.12 -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

The last line must print **True** and the GPU name. If it prints False, the CPU build was installed: uninstall torch/torchvision and repeat the first line.

B3. Check everything works

```
py -3.12 -m pytest tests/ -q
py -3.12 scripts/download_data.py --synthetic
py -3.12 scripts/verify_dataset.py --root data/mvtec_loco
```

Expect **285 passed** and *PASSED* from the verifier. The synthetic set writes **data/mvtec_loco/screw_board/defect_subtypes.json**, which powers the removal/addition breakdown.

B4. Before any long run: three rules

- **Add data.num_workers=0** if Windows ever reports "**paging file is too small**". Each loader worker loads PyTorch's CUDA libraries separately and can exhaust virtual memory.
- **Long runs survive interruptions.** Re-running the same **run_ablations.py** command with **--no-retrain** reuses every finished model (finished models are only re-evaluated, not retrained). Only the model in progress is lost.
- **Don't let the laptop sleep** mid-run; CUDA work often does not survive it.

Watching progress from a second Command Prompt:

```
findstr /C:"| seed" /C:"Training finished" /C:"-> " outputs\<experiment>\<category>\ablations.log
powershell -Command "Get-Content outputs\<experiment>\<category>\ablations.log -Wait -Tail 5"
```

B5. fp32 target_std check (~15 min on a fast GPU)

On the GTX 1650, **target_std** settled at ~0.1 instead of the README's ~0.5, while detection stayed healthy. The suspected cause is emulated bf16 on that GPU. Two short runs settle it:

```

py -3.12 train.py --config configs/ablations.yaml --name tstd_amp `
--set train.save_interval=0
py -3.12 train.py --config configs/ablations.yaml --name tstd_fp32 `
--set train.save_interval=0 train.amp=false

foreach ($run in "tstd_amp", "tstd_fp32") {
  "--- $run"
  Select-String -Path "outputs\$run\screw_board\train.log" -Pattern "target_std=[0-9.]+\" |
    ForEach-Object { $_.Matches[0].Value }
}

```

What you see	What it means
fp32 stays ~0.4–0.5, amp ~0.1	Precision causes it. Note in README; consider forcing fp16 on GPUs without native bf16.
Both ~0.1	Not precision: a difference in this torch version or the run setup. Detection is still fine; just record it.
Both ~0.4–0.5	Only the GTX 1650 shows the effect. Record it as hardware-specific.

B6. Get the real MVTec LOCO AD dataset (manual, license-gated)

- Register and download from the official MVTec website (search "MVTec LOCO AD"). It is free for non-commercial research. Accept the license yourself; do not redistribute the data.
- Extract it so the five category folders sit directly under **data/mvtec_loco_real**: breakfast_box, juice_bottle, pushpins, screw_bag, splicing_connectors.

```

py -3.12 scripts/verify_dataset.py --root data/mvtec_loco_real --strict-loco
py -3.12 scripts/prepare_loco.py --src data/mvtec_loco_real --dst data/mvtec_loco_256
py -3.12 scripts/verify_dataset.py --root data/mvtec_loco_256 --strict-loco

```

The verifier must report PASSED. **prepare_loco.py** pre-resizes to 256 px (about 12× faster loading; identical tensors). Every command below uses **data/mvtec_loco_256**.

B7. Studies 8 + 9 on real LOCO pushpins (the main pending result)

Study 8 trains 4 λ arms $\times$ 3 seeds; it also produces the slot-enabled checkpoints that Study 9 (the cardinality channel) and Study 7 below re-score. Run them in order:

```

$common = @("experiment.name=real_phase3bc", "data.root=data/mvtec_loco_256",
"data.category=pushpins", "train.save_interval=0")

py -3.12 run_ablations.py --config configs/ablations.yaml --study hierarchical `
--seeds 0,1,2 --no-retrain --set @common
py -3.12 run_ablations.py --config configs/ablations.yaml --study cardinality `
--seeds 0,1,2 --no-retrain --set @common

```

Results: **outputs\real_phase3bc\pushpins\hierarchical_study.csv** and **cardinality_study.csv**, plus **ablations.log**. On real data the removal/addition columns read n/a, because LOCO ships no subtype manifest; the logical column is the one to read.

B8. Study 7 on real pushpins: signed vs absolute deviation

Phase 1 measured logical AUROC **0.449 $\pm$ 0.003** (below chance) on pushpins with signed scoring. This is the decisive test of the fix. It re-scores the three $\lambda = 0$ checkpoints from B7, one seed at a time:

```

foreach ($s in 0,1,2) {
  py -3.12 run_ablations.py --config configs/ablations.yaml --study deviation --seeds $s `
  --checkpoint "checkpoints/real_phase3bc/hier_lambda_0/seed_$s/pushpins/final.pt" `
  --set "experiment.name=real_study7_seed$s" data.root=data/mvtec_loco_256 data.category=pushpins
}

```

Compare **logical_auroc** for dev_signed vs dev_absolute across the three seed folders (**outputs\real_study7_seed*\pushpins\deviation_study.csv**). Above 0.5 with absolute means the Phase 1 mechanism is confirmed on real data.

B9. The other four categories (is below-chance logical universal?)

```
foreach ($c in "breakfast_box", "juice_bottle", "screw_bag", "splicing_connectors") {
    $set = @("experiment.name=real_phase3bc", "data.root=data/mvtec_loco_256",
            "data.category=$c", "train.save_interval=0")
    foreach ($study in "hierarchical", "cardinality") {
        py -3.12 run_ablations.py --config configs/ablations.yaml --study $study `
            --seeds 0,1,2 --no-retrain --set @set
    }
}
```

This is the longest step. If time is short, run only **splicing_connectors** and **breakfast_box**: their components are visually distinct, which is the one condition under which the cardinality head might behave differently from the synthetic set.

B10. Slot isolation gate on real data (Phase 3a re-check)

On synthetic data, slots grouped by component *type* (all screws in one slot). Real categories with distinguishable parts may group by *instance*, which would change what a count can see.

```
$ck = "checkpoints/real_phase3bc/hier_lambda_0/seed_0/splicing_connectors/final.pt"
py -3.12 train_slots.py --checkpoint $ck --num-slots 4,6,8,12 --epochs 150 --lr 8e-4 `
    --set data.root=data/mvtec_loco_256 data.category=splicing_connectors
py -3.12 visualize.py --checkpoint $ck --mode slots `
    --slot-checkpoint outputs/phase3a_slots/splicing_connectors/slots_6.pt `
    --set data.root=data/mvtec_loco_256 data.category=splicing_connectors
```

Read **outputs\phase3a_slots\splicing_connectors\gate_results.json** (PASS/FAIL per slot count) and look at the rendered slot maps: does each connector get its own slot?

B11. Open research item: scoring removed objects

The one problem no arm fixed: removed content scores 0.47–0.58 while added content scores ~1.0. This needs new code, not just GPU time. The idea recorded in the Phase 3b/3c summary:

- Fit, on normal images only, a per-position **occupancy prior**: how often each patch is covered by a non-background slot.
- At test time, score **|prior – observed occupancy|** per region. An emptied mounting point is then a large deviation in its own right, independent of how easy it is to predict.
- Evaluate it as a new **anomaly.cardinality**-style channel on synthetic data first; the **removal_auroc** column shows immediately whether it works.

B12. Time budget

Step	GTX 1650 (measured)	RTX 4060-class (estimate)
B3 tests + synthetic data	~2 min	~1 min
B5 two target_std runs	~25 min	~8 min
B7 Studies 8+9, one category	~6.5 h on synthetic	2–3 h (LOCO has more images)
B8 Study 7, 3 seeds	~1.5 h	~30 min
B9 four more categories	1–2 days	8–12 h
B10 slot gate	~1 h	~20 min

Evaluation (the 299-window sweep) dominates: about 9 minutes per model on the GTX 1650.

Part C — Bring the results back and finish

C1. Teammate: package the results

```
cd C:\work\logical-JEPA
Compress-Archive -Force -DestinationPath logical_jepa_gpu_results.zip -Path `
  outputs\tstd_amp, outputs\tstd_fp32, outputs\real_phase3bc, outputs\real_study7_seed0, `
  outputs\real_study7_seed1, outputs\real_study7_seed2, outputs\phase3a_slots
```

Only include folders that exist. Send the zip; do **not** send **checkpoints/** (hundreds of MB, regenerable) or anything from **data/** (licensed).

C2. Write up and push (either laptop)

- Unzip into the repo's **outputs/** folder.
- Write **outputs/phase6_real/SUMMARY.md** in the same style as **outputs/phase3bc/SUMMARY.md**: setup, tables as mean $\pm$ std over seeds, what was confirmed, what was retracted, caveats, and exact commands to reproduce.
- Update **README.md**: replace the synthetic-only caveats with the real numbers, and resolve Research question 2 and the Study 7 note.
- Commit **only** the summary and README, then push (Part A, steps A3–A5).

```
git add outputs/phase6_real/SUMMARY.md README.md
git commit -m "Record real MVTEC LOCO results for Studies 7-9"
git push origin HEAD:vikram
```

C3. Merge vikram into main (repository owner's decision)

The owner already merged **PR #1** (commit 81f6062, the Phase 3b/3c code). The later commits on vikram (5cbd3bb results + checkpoint-loading fix, 3ad6f84 demo card, and anything from C2) still need a **second pull request**: on GitHub, open *Pull requests* → *New*, base **main**, compare **vikram**. Command-line alternative, only with the owner's agreement:

```
git fetch origin
git checkout main
git pull origin main
git merge --no-ff origin/vikram -m "Merge Phase 3b/3c and real-data results"
py -3.12 -m pytest tests/ -q
git push origin main
```

Troubleshooting

Symptom	Fix
OSError WinError 1455 "paging file is too small"	Add data.num_workers=0 to the --set list, close other Python processes, or enlarge the Windows paging file / free disk space on C:.
Run stopped midway (crash, reboot, closed window)	Re-run the identical command; --no-retrain keeps finished models. Only the model in progress is lost.
RuntimeError: Unexpected key(s) in state_dict ... slots....	You are on an old commit. git checkout vikram ; git reset --hard origin/vikram (the fix is in 5cbd3bb).
anomaly.cardinality=add needs a checkpoint trained with a cardinality head	Use a checkpoint from a hier_lambda_* arm, not from the masking study.
torch.cuda.is_available() is False	CPU torch is installed. Reinstall with the cu126 index URL from B2.
target_std warning in the training log	Values ~0.1 were measured as harmless here (B5 explains). Only below 0.05 is a real warning.
Push rejected (403 / permission denied)	The GitHub account needs collaborator access to Ratnachand04/logical-JEPA.

Symptom	Fix
Push rejected (non-fast-forward)	git pull --rebase origin vikram , run tests, push again. Never force-push over others' work.

Final checklist

#	Task	Where	Done
1	Switch back to the work branch; verify pushed; tests pass (A1, A4, A5)	This laptop	
2	Clone, install, 285 tests pass (B1–B3)	Teammate	
3	fp32 target_std check (B5)	Teammate	
4	Download and verify real LOCO; pre-resize (B6)	Teammate	
5	Studies 8+9 on pushpins (B7)	Teammate	
6	Study 7 on pushpins (B8)	Teammate	
7	Other four categories (B9)	Teammate	
8	Slot gate on real data (B10)	Teammate	
9	Removal-scoring idea (B11, optional research)	Either	
10	Zip results and send back (C1)	Teammate	
11	Phase 6 summary + README, commit, push (C2)	Either	
12	Second pull request vikram → main (C3)	Owner	